CloudVault — Mini Cloud File Storage System

Project blueprint · FastAPI · PostgreSQL · Supabase Storage · React · 5 days

1. Project overview

CloudVault is a simplified Google Drive / Dropbox. Every registered user gets a private storage space to upload, organise, download and share files through secure links. The goal is to understand how cloud storage really works: file handling, metadata versus bytes, ownership, permissions, quotas, sharing and secure delivery.

Journey: register → log in → dashboard → upload (validated, bytes in a private Supabase Storage bucket, metadata in PostgreSQL) → view, search, rename, move, delete → create a share link → another person opens it and receives the file only if the link is valid and permits it.

Scope statement. Supabase provides PostgreSQL and private object storage. FastAPI remains responsible for authentication, validation, ownership checks, quota enforcement, sharing, and secure file access. The React frontend never receives the Supabase service-role key and never directly accesses private storage without backend authorization.

Key decisions

Topic	Decision
Max upload size	100 MB (MAX_UPLOAD_BYTES)
Default quota	2 GB = 2,000,000,000 bytes (decimal units: 1 GB = 1,000,000,000 bytes)
Duplicate names	Same name in the same folder for the same user → 409 (upload, create, rename, move)
Other people's resources	Always 404, never 403
Error format	JSON with a machine-readable code (e.g. QUOTA_EXCEEDED) and a human message
Root folder	Virtual: folder_id / parent_id = NULL
Deletion	Permanent (no trash in v1)
Previewable types	PDF, PNG, JPG/JPEG, TXT; everything else is download-only
Search	With q set, searches all of the user's files, not just the current folder
Share expiry	Required; default 7 days, maximum 30
Share view permission	Only for previewable types (otherwise 422). ZIP is not previewable, so a view share on a ZIP returns 422 and ZIP files support download shares only
Authentication	Custom JWT authentication only. Supabase is used for PostgreSQL and Storage, not for Supabase Auth, so there is a single user/authentication system
File bytes	Private Supabase Storage bucket; the object key is stored in files.storage_key
Migrations	Alembic is the only migration system. The schema is never changed by hand in the Supabase dashboard or SQL editor
Logout	Client discards the token; tokens expire after 60 minutes

2. Functional requirements

Module A — Authentication. Register (name, email, password), login, GET /me, Argon2 hashing via pwdlib, JWT bearer tokens, protected routes. Passwords are 8–128 characters. Unknown email and wrong password return the same error (verify a dummy hash for unknown emails to equalise timing). Every file, folder and share is reachable only through ownership-checked queries.

Module B — Files. Upload, download, preview, rename, delete, search by name, sort (name, date, size), filter by type, view details.

Type	Extensions	Server-assigned MIME	Content check
Documents	PDF	application/pdf	begins with %PDF
Documents	DOCX	application/vnd.openxmlformats-officedocument.wordprocessingml.document	ZIP container recognised as DOCX by filetype
Documents	TXT	text/plain	valid UTF-8, no NUL bytes in first 8 KB
Images	PNG	image/png	PNG signature
Images	JPG, JPEG	image/jpeg	JPEG signature
Archives	ZIP	application/zip	ZIP signature (never extracted server-side)
Data	CSV, JSON	text/csv, application/json	UTF-8 text check

The extension must agree with the sniffed content. The client's Content-Type is ignored.

Module C — Folders. Create, rename, delete, nest, upload into a folder, move files between folders. (Moving folders is not in scope; if you add it later it needs a cycle check.)

Module D — Sharing. Unguessable link per file, expiry, view or download permission, revocation, optional password. Checks run on every access (§7.4). View shares are allowed only for previewable types; ZIP files can only be shared with the download permission.

Module E — Storage. Per-user quota. used_bytes is stored in the database and maintained transactionally on upload and delete. The bucket is never scanned to compute usage. Uploads that would exceed the quota are rejected.

3. Architecture

- React shows files and folders, sends FormData, shows progress and usage. It never talks to Supabase and never receives the Supabase service-role key.
- FastAPI is the controller: routers are thin, services hold business logic, SupabaseStorage is the only code that talks to Supabase Storage.
- PostgreSQL (hosted on Supabase) stores metadata only. The bytes live in a private Supabase Storage bucket under server-generated object keys.
- The bucket is private. Files are reachable only through backend authorization (the backend streams the object after the ownership or share check). Short-lived signed URLs may be generated by SupabaseStorage when needed, never without an authorization check first.

Authorization model. The backend is the only client of Supabase. It connects to PostgreSQL through a protected server-side connection and to Storage with the service-role key. All authorization is therefore enforced by the backend ownership checks (§7.5), not by database policies. Row Level Security is enabled on all four tables with no policies, so the Supabase Data API (anon and authenticated roles) cannot read or write them; the backend's database role bypasses RLS. The Storage bucket has no policies for anon or authenticated users, so nobody can read another user's objects (or any object) except through the backend. If the frontend ever accesses Supabase directly, RLS policies must be written and tested first.

Deployment. The application no longer depends on persistent local disk. The backend can run on a container, VM or serverless-compatible platform. Supabase Storage is the durable file layer. The Supabase URL, bucket name, service-role key and database URL are configured through environment variables.

Stack: Python 3.11+, FastAPI, Uvicorn, SQLAlchemy 2.0 (sync) + psycopg, Alembic, PostgreSQL (Supabase), Supabase Storage (REST API via httpx), pwdlib[argon2], PyJWT, filetype, slowapi, pytest + httpx. React + Vite, Tailwind v4, Axios, React Router, lucide-react.

Example metadata row: id 101 · original_name Resume.pdf · storage_key 5/a81f92….bin · size 250000 · mime_type application/pdf · user_id 5 · folder_id 12.

4. Database design (four tables)

Table	Columns
users	id PK · name · email (lower-cased, unique) · password_hash · storage_quota BIGINT default 2000000000, CHECK ≥ 0 · used_bytes BIGINT default 0, CHECK ≥ 0 · created_at TIMESTAMPTZ
folders	id PK · user_id → users · parent_id · name · created_at. UNIQUE (id, user_id). FK (parent_id, user_id) → folders (id, user_id)
files	id PK · user_id → users · folder_id · original_name · extension · storage_key UNIQUE (Supabase object key, server-generated: {user_id}/{uuid hex}.bin) · size BIGINT CHECK ≥ 0 · mime_type · checksum (SHA-256 hex) · created_at · updated_at. FK (folder_id, user_id) → folders (id, user_id)
shares	id PK · file_id → files ON DELETE CASCADE · token_hash UNIQUE (SHA-256 hex) · permission CHECK in ('view','download') · password_hash NULL · expires_at TIMESTAMPTZ NOT NULL · revoked_at NULL · created_at

Relationships: a user has many folders and files; a folder has many files and child folders (self-reference via parent_id: Documents → NULL, Work → Documents, Reports → Work); a file has many shares.

The composite foreign keys use the default NO ACTION (checked at the end of the statement, so a recursive delete in one statement works). Because user_id is part of the key, the database itself refuses a file in another user's folder. A NULL folder_id means root and skips the check.

```
-- Uniqueness that also works at root (NULLs are distinct in PostgreSQL)
CREATE UNIQUE INDEX uq_folders_root ON folders (user_id, name) WHERE parent_id IS NULL;
CREATE UNIQUE INDEX uq_folders_child ON folders (user_id, parent_id, name) WHERE parent_id IS NOT NULL;
CREATE UNIQUE INDEX uq_files_root ON files (user_id, original_name) WHERE folder_id IS NULL;
CREATE UNIQUE INDEX uq_files_child ON files (user_id, folder_id, original_name) WHERE folder_id IS NOT NULL;
```

Other indexes: files (user_id, folder_id), files (user_id, created_at DESC), folders (user_id, parent_id), shares (file_id).

Rule	Enforced by
Every file has an owner	user_id NOT NULL
File and folder belong to the same user	composite FK
Can't move a file into another user's folder	composite FK + ownership check (404)
Share tokens are unique	unique index on token_hash
Sizes can't be negative	CHECK constraints
Non-empty folders aren't deleted by accident	FK + 409 unless ?recursive=true
Tables are not reachable through the Supabase Data API	RLS enabled on all four tables, no policies

Migrations. Alembic is the authoritative migration system. Set it up on Day 1 and create one migration per day. Never modify the schema manually in Supabase (dashboard or SQL editor); every schema change, including enabling RLS, goes through an Alembic migration. Autogenerate doesn't reliably detect CHECK constraints or partial-index conditions, so read every migration before applying it. Import all models in models/__init__.py so Alembic sees them.

5. Backend API

All routes are under routers; every private route requires Authorization: Bearer <token>.

Method	Endpoint	Notes
POST	/api/auth/register	JSON: name, email, password → 201
POST	/api/auth/login	Form fields username (email) and password (so Swagger's Authorize button works) → access_token
GET	/api/auth/me	Name, email, quota

Method	Endpoint	Notes
POST	/api/files/upload	Multipart: file + optional folder_id form field → 201
GET	/api/files	Query: folder_id, q, type (documents, images, archives, data), sort (name, created_at, size), order, limit (max 100), offset → items, total, limit, offset
GET	/api/files/{id}	Details (never includes storage_key or any path)
GET	/api/files/{id}/download	?inline=true for preview (previewable types only)
PATCH	/api/files/{id}	Body: name and/or folder_id; null folder_id = move to root
DELETE	/api/files/{id}	204
POST	/api/folders	name, parent_id
GET	/api/folders	?parent_id= lists children (omitted = root)
GET	/api/folders/{id}	Folder, breadcrumbs, child folders and files
PATCH	/api/folders/{id}	Rename
DELETE	/api/folders/{id}	409 if not empty unless ?recursive=true
POST	/api/shares	file_id, permission, expiry, optional password → returns the token once. 422 for permission view on a non-previewable file (including ZIP)
GET	/api/shares	List the caller's shares (metadata only)
GET	/api/shares/{id}	Share settings
DELETE	/api/shares/{id}	Revoke (sets revoked_at)
GET	/api/public/share/{token}	No auth. Metadata, or 401 PASSWORD_REQUIRED
GET	/api/public/share/{token}/content	No auth. Delivers the file (inline for view, attachment for download)
GET	/api/storage/usage	used_bytes, quota_bytes, remaining_bytes, percent_used

Status codes: 400/422 validation, 401 unauthenticated, 404 not found or not yours, 409 duplicate name or non-empty folder, 413 too large or quota exceeded (distinguished by code), 429 rate limited, 502 storage unavailable (STORAGE_UNAVAILABLE).

6. Project structure

```
CloudVault/
├── backend/
│   ├── app/
│   │   ├── main.py        # app, CORS, middleware, routers
│   │   ├── config.py      # pydantic-settings, reads .env
│   │   ├── database.py    # engine, SessionLocal, get_db
│   │   ├── errors.py      # AppError(status, code, message)
│   │   ├── cleanup.py     # scheduled orphan-object cleanup entry point
│   │   ├── models/        # user, folder, file, share (+ __init__ importing all)
│   │   ├── schemas/       # request/response models (no storage_key)
│   │   ├── routers/       # auth, files, folders, shares, public, storage
│   │   ├── services/      # file_service, folder_service, share_service,
│   │   │                  # supabase_storage_service, cleanup_service
│   │   └── utils/         # security.py, dependencies.py, validation.py
│   ├── migrations/
│   ├── tests/
│   ├── .env.example
│   └── requirements.txt
├── .gitignore             # .env, venv/, node_modules/
└── frontend/src/{components,pages,services,hooks,context}
```

Routers receive HTTP, services hold logic, models are tables, schemas validate I/O, dependencies.py provides the DB session and current user. There is no storage/ directory: nothing is written to local disk by the application.

7. Core algorithms

7.1 Upload

1. Authenticate. Resolve folder_id with the ownership helper (404 if not yours). Sanitise the name (strip path parts and control characters, max 255 chars) and check the extension allowlist. Reject early if Content-Length exceeds the limit.
2. Stream the body through FastAPI in 1 MB chunks, counting bytes (over the limit → 413 FILE_TOO_LARGE), updating the SHA-256, and keeping the first 8 KB for sniffing.
3. Sniff the content and compare with the extension (mismatch → 422 UNSUPPORTED_TYPE).
4. Open the DB transaction and reserve quota atomically. No row updated → 413 QUOTA_EXCEEDED.
5. Generate the object key on the server ({user_id}/{uuid4 hex}.bin) and upload the validated file to the private Supabase bucket.
6. Only after the storage upload succeeds, insert the metadata row and flush(). A duplicate name raises → 409.
7. commit(). On any exception: rollback() (this also undoes the quota reservation) and delete the uploaded Supabase object.

```
CHUNK = 1024 * 1024
sha, size, head = hashlib.sha256(), 0, b''
while chunk := upload.file.read(CHUNK):
    size += len(chunk)
    if size > settings.MAX_UPLOAD_BYTES:
        raise AppError(413, 'FILE_TOO_LARGE')
    if len(head) < 8192:
        head += chunk[:8192 - len(head)]
    sha.update(chunk)
# atomic quota reservation: no check-then-write race
row = db.execute(
    update(User)
    .where(User.id == user.id, User.used_bytes + size <= User.storage_quota)
    .values(used_bytes=User.used_bytes + size)
    .returning(User.used_bytes)
).first()
if row is None:
    raise AppError(413, 'QUOTA_EXCEEDED')
```

The quota UPDATE keeps its row lock until commit, so two uploads by the same user are serialised during the storage upload; this is what makes the reservation atomic. A crash between step 5 and step 7 leaves an object without metadata; the orphan cleanup (§7.8) removes it.

7.2 Delete (file and folder)

In one transaction: DELETE FROM files WHERE id = :id AND user_id = :uid RETURNING size, storage_key, then UPDATE users SET used_bytes = used_bytes - :size, then commit. Only after the commit succeeds, delete the corresponding object from Supabase Storage (a missing object is ignored). If the object deletion fails, log the failure; the object is then an orphan and the cleanup task (§7.8) removes it. A recursive folder delete finds descendants with a recursive CTE, sums their file sizes, deletes files and then folders in one statement each, decrements usage once, commits, then deletes the objects. Never the other way round: a DB row pointing at a missing object is worse than an orphaned object.

7.3 Download and preview

Ownership helper (or share check) → SupabaseStorage streams the object, validating the key first → FastAPI returns it with media_type=file.mime_type and Content-Disposition inline or attachment (original name), Content-Length, and X-Content-Type-Options: nosniff. Inline preview is allowed only for PDF, PNG, JPG/JPEG and TXT; everything else is an attachment. The bucket is never exposed to the browser. If the object is missing, log it and return 404 without keys or paths in the message. Short-lived signed URLs are available from SupabaseStorage for cases where streaming is not suitable, but only after the same authorization check.

7.4 Public share access

1. Hash the presented token with SHA-256 and look it up by token_hash.
2. Not found, revoked, expires_at <= now() (UTC), file deleted, or object missing in storage → the same 404 SHARE_NOT_FOUND (log the real reason internally).
3. Password set? Verify X-Share-Password with pwdlib; missing or wrong → 401 PASSWORD_REQUIRED.

4. Permission: view → inline; download → attachment.
5. Stream the object. Add X-Content-Type-Options: nosniff, Referrer-Policy: no-referrer and Cache-Control: no-store.

Tokens are generated with secrets.token_urlsafe(32). Since the token is high-entropy, a plain SHA-256 is the right hash (a slow password hash would add nothing).

7.5 Ownership helper

Every router obtains files, folders and shares only through one helper, so a check can't be forgotten.

```
def get_owned_file(db, user, file_id):
    f = db.scalar(select(File).where(File.id == file_id, File.user_id == user.id))
    if f is None:
        raise AppError(404, 'FILE_NOT_FOUND')
    return f
# get_owned_folder and get_owned_share (join to File.user_id) work the same way
```

7.6 PATCH semantics

```
changes = body.model_dump(exclude_unset=True)   # distinguishes omitted from null
if 'folder_id' in changes:                      # null here means move to root
    target = None if changes['folder_id'] is None else get_owned_folder(db, user, changes['folder_id'])
```

Rename keeps an allowed extension of the same type and re-checks the duplicate-name rule; a collision raises 409.

7.7 Object-key validation

Object keys are always generated by the server and never come from the client. Before any Supabase call, SupabaseStorage checks the key against the pattern digits/32-hex-characters.bin; anything else is rejected. Display names never become keys.

7.8 Orphan-object cleanup

Supabase Storage may contain objects without metadata if the application crashes between upload and database commit, or if an object deletion failed after the database commit. A scheduled cleanup task (python -m app.cleanup, run hourly by cron or the platform's scheduler) lists the bucket, then loads the known storage_key values from PostgreSQL, and deletes every object that has no matching row and is older than a one-hour grace period (so uploads still in progress are not touched). The bucket is listed before the database is queried, so an object committed in between is never mistaken for an orphan.

8. Security requirements

Risk	Control
Unauthorized access (IDOR)	Ownership helper on every query; 404 for foreign resources
Malicious uploads	Extension allowlist + magic-byte sniffing + server-assigned MIME; ZIPs are never extracted
Path traversal / key injection	Object keys are server-generated and validated against a strict pattern; display names never become keys
Direct bucket access	Bucket is private with no policies for anon/authenticated users; files are delivered only after backend authorization (streaming or short-lived signed URL)
Service-role key leakage	Key only in backend environment variables; never sent to the frontend, never logged, never committed
Database exposure	RLS enabled on all tables with no policies; authorization enforced by backend ownership checks; database credentials only in .env
Stored XSS / sniffing	nosniff, inline only for PDF/PNG/JPEG/TXT, attachment otherwise. (Production-grade answer: serve user files from a separate domain.)
Password theft	Argon2 via pwdlib; length cap 8–128
Weak or leaked tokens	256-bit tokens, only the hash stored, password in a header, no-referrer

Risk	Control
Expired or revoked links	Checked on every access with UTC now(); uniform 404
Storage abuse	Atomic per-user quota; 100 MB upload cap
Large uploads	Content-Length check + proxy cap + chunked processing
SQL injection	SQLAlchemy bound parameters; whitelisted sort fields; escaped ILIKE wildcards
Brute force	slowapi on login, register and public share routes
JWT misuse	exp claim, pinned algorithm, strong secret from env

JWT storage. A JWT in localStorage can be stolen if an XSS vulnerability occurs. For production, prefer secure, HttpOnly cookies. Version one keeps the access token in localStorage (simple), and therefore enforces strict security headers on every response, validates all input, and serves the frontend with a Content-Security-Policy where possible. This is also why the file-serving headers above matter.

Also: secrets only in environment variables; never commit .env; database passwords are URL-encoded in the connection string; CORS restricted to FRONTEND_ORIGIN; response schemas never expose storage_key or paths; use HTTPS in production; never log tokens, passwords or share URLs. Tests never use the production Supabase database.

9. Frontend

Dark-theme dashboard in React + Tailwind.

Page	Elements
Login, Register	Email and password (Register adds name)
Dashboard	Storage overview, upload button, recent files (sort=created_at&order=desc&limit=5)
My Files	One explorer: breadcrumbs, nested folders, grid/list toggle, search, type filter, sort, pagination
Shared Files	Share list (file, permission, expiry, status) and revoke; the link is shown once at creation with a copy button
Settings	Read-only account and storage information
Public Share (/share/:token)	No login; password prompt if needed; view or download

Layout: left sidebar (Dashboard, My Files, Shared, Settings), top bar (search, profile), upload modal with drag-and-drop, progress bar and status.

Behaviour rules

1. One Axios instance adds the bearer token and, on a 401, clears the token and redirects to Login. A ProtectedRoute wraps private pages; on load, validate the token with /auth/me.
2. Upload: build FormData, don't set Content-Type manually, use onUploadProgress, show "Processing…" at 100% until the response arrives, then update the list and refresh /storage/usage. Each file is its own request, run one at a time.
3. Preview and download: request as a blob from the backend with Axios, create an object URL, and revoke it when finished. The frontend never contacts Supabase. (Large downloads are held in memory; acceptable for a 100 MB cap.)
4. Refresh storage usage after every upload and delete. Show loading, empty and error states, with messages taken from the backend message.
5. One formatBytes helper using decimal units everywhere.
6. The share dialog disables the View permission for non-previewable files, including ZIP.

10. Testing

Use pytest + FastAPI TestClient against a dedicated local PostgreSQL test database (not SQLite, and never the Supabase database), with a dependency override for the DB session. Storage tests use a dedicated private test bucket (cloudvault-test) and separate TEST_ environment variables; the suite refuses to start if the database URL points at Supabase or the bucket name does not end in -test.

Test	Expected
Register; duplicate email; email case variants	201; 409; 409
Login with wrong password / unknown email	Same 401 for both
Expired or tampered JWT	401
Upload valid PDF to Supabase Storage	201, row created, object exists in the bucket under {user_id}/...bin, used_bytes increased
Upload oversized file	413 FILE_TOO_LARGE, no object stored
Upload unsupported type; exe renamed to .pdf	422
Upload with spoofed Content-Type	Ignored; decided by content
Filename like ../../etc/passwd	Sanitised; object key is server-generated
Upload beyond quota	413 QUOTA_EXCEEDED, no object stored
Two parallel uploads that together exceed quota	Exactly one succeeds
Failed database insert after successful storage upload	Uploaded object deleted, usage unchanged
Download own file	200, correct bytes and checksum
Download / rename / move / delete another user's file	404
Cross-user object access	Other user gets 404 and the object is untouched; keys carry the owner's id prefix
Private bucket access	Unauthenticated and public-URL requests for an object are denied
Signed URL expiration	URL works before expiry, is rejected after
Move file into another user's folder	404
Rename to a disallowed extension; rename to an existing name	422; 409
Delete a file	Row and object removed, usage decreased
Storage deletion failure	204, row removed, usage decreased, failure logged; cleanup removes the leftover object
Orphan-object cleanup	Object without row (older than the grace period) deleted; objects with rows kept
Delete non-empty folder; with recursive=true	409; contents and objects removed, usage correct
Invariants after every test	used_bytes == SUM(files.size) for each user; set of storage_key values == set of objects in the bucket
RLS policies	Not applicable in v1: the frontend never accesses Supabase directly, tables have RLS enabled with no policies, and authorization is the backend's ownership checks. Add policy tests only if direct access is introduced
Create share for another user's file; read or revoke another user's share	404
Access share: valid / expired / revoked / file deleted	200 / 404 / 404 / 404
Password-protected share: none / wrong / correct	401 / 401 / 200
view share: response disposition	inline
view share on a ZIP; download share on a ZIP	422 at creation; 201
Large file (e.g. 95 MB passes, 101 MB rejected)	Memory stays flat
Rate limit on login	429 after threshold

11. Five-day roadmap (vertical slices)

Each day finishes backend, tests and the matching UI together, so problems surface early and Day 5 is only final hardening. Five days is realistic if you already know FastAPI and React; if you are learning both, plan 7–8 days.

Day 1 — Foundation and authentication. Project skeleton, config.py, database, Alembic with the first migration containing all tables, constraints and RLS enablement, error handler, register, login, /me, JWT dependency. Frontend: Vite + Tailwind, Login and Register pages, auth context, ProtectedRoute, Axios instance. Tests: auth. Target: register, log in and reach a protected page.

Day 2 — Upload and download engine. SupabaseStorage service, streaming upload, validation and sniffing, SHA-256, atomic quota reservation, download and preview, file details, list, rename, delete with usage updates. Frontend: upload modal with progress, file list, download and preview. Tests: upload, download, ownership, quota, failure cleanup, private bucket, signed URL. Target: securely upload, view, download and delete your own files while usage stays exact.

Day 3 — Folders, search and storage view. Folder CRUD with nesting, recursive delete, move files, search, filter, sort, pagination, storage-usage endpoint. Frontend: explorer with breadcrumbs, storage bar, filters. Tests: folders, moves, cross-user folder attacks, concurrency. Target: organise files and see accurate usage.

Day 4 — Sharing. Share creation, hashed tokens, expiry, permissions (ZIP download-only), password option, revocation, list endpoint, public routes. Frontend: Shared Files page, create-share dialog, public Share page. Tests: all share cases. Target: share a file safely and revoke it.

Day 5 — Hardening and polish. Rate limiting, security headers, CORS check, orphan-object cleanup task, large-file test, full cross-user test pass, loading and error states, README. Target: a demonstrable, tested CloudVault.

12. Setup

```
mkdir CloudVault && cd CloudVault
mkdir backend && cd backend
python3 -m venv venv
source venv/bin/activate          # Windows PowerShell: venv/Scripts/Activate.ps1
pip install fastapi uvicorn 'sqlalchemy>=2' 'psycopg[binary]' alembic python-multipart \
    pyjwt 'pwdlib[argon2]' pydantic-settings email-validator filetype slowapi httpx
pip install pytest
pip freeze > requirements.txt     # pin versions
mkdir -p app/models app/schemas app/routers app/services app/utils tests
uvicorn app.main:app --reload --port 8000
```

On Windows without bash, create the directories manually or use WSL. API docs: http://127.0.0.1:8000/docs.

In the Supabase dashboard, create a private bucket (Public bucket switched off) named cloudvault and a second private bucket named cloudvault-test for the tests. Add no storage policies for either bucket. Set the bucket file size limit to at least MAX_UPLOAD_BYTES (check your Supabase plan's per-file limit).

.env.example:

```
# Supabase: Connect > Session pooler string (port 5432), with the scheme changed to postgresql+psycopg://
# URL-encode special characters in the password. Never commit .env.
DATABASE_URL=postgresql+psycopg://postgres.REF:PASSWORD@aws-0-REGION.pooler.supabase.com:5432/postgres?sslmode=require
# Supabase Storage (server-side only; never expose the service-role key to the frontend)
SUPABASE_URL=https://REF.supabase.co
SUPABASE_SERVICE_ROLE_KEY=paste-the-service-role-key-here
SUPABASE_BUCKET=cloudvault
SECRET_KEY=generate-with: python -c 'import secrets; print(secrets.token_urlsafe(48))'
ACCESS_TOKEN_MINUTES=60
MAX_UPLOAD_BYTES=104857600
DEFAULT_QUOTA_BYTES=2000000000
FRONTEND_ORIGIN=http://localhost:5173
```

Frontend:

```
cd CloudVault
npm create vite@latest frontend -- --template react
cd frontend && npm install
npm install axios react-router-dom lucide-react
npm install -D tailwindcss @tailwindcss/vite
```

For Tailwind, follow the current Vite plugin instructions (add the plugin to vite.config.js and @import 'tailwindcss'; to your CSS).

13. Completion checklist (33 items)

Foundation

- ☐ FastAPI with settings from environment
- ☐ PostgreSQL + Alembic migrations (Alembic is the only migration system)
- ☐ All models with constraints and indexes
- ☐ Registration and login
- ☐ JWT auth and protected routes

Files

- ☐ Streaming upload with size limit
- ☐ Content validation and SHA-256 checksum
- ☐ Download and preview
- ☐ Rename and delete
- ☐ Search, filter, sort, pagination

Folders

- ☐ Create and nest folders
- ☐ Rename folders
- ☐ Move files
- ☐ Delete folders (409 or recursive)

Storage

- ☐ Private Supabase Storage bucket, bytes never on local disk
- ☐ Atomic quota enforcement
- ☐ Usage endpoint and dashboard bar
- ☐ Orphan-object cleanup

Sharing

- ☐ Create link (token shown once)
- ☐ Expiry
- ☐ View / download permission (ZIP: download only)
- ☐ Optional password
- ☐ Revoke
- ☐ Public share page

Frontend and testing

- ☐ Login and register pages
- ☐ Dashboard and explorer
- ☐ Upload progress
- ☐ Shared Files page and Settings
- ☐ Cross-user access tests
- ☐ Share expiry, revoke and password tests
- ☐ Quota concurrency and large-file tests
- ☐ Loading and error states; rate limiting and security headers

14. Build in two stages

Stage 1 — Must complete: everything in this blueprint, including sharing.

Stage 2 — Future upgrades (out of scope here): file versioning, duplicate detection using the stored checksum, recycle bin, and another storage provider (for example AWS S3) by adding a second implementation of the storage service.

Still out of scope: real-time collaboration, simultaneous editing, device sync, AI document analysis, video streaming, distributed storage, microservices, Kubernetes, complex role management.